

GO PROGRAMMING MASTER COURSE

Week 3, Day 1

Control flow, slices, maps, and just enough functions — the tools you will use to filter students later.

Umesh Indrajith · BSc Eng (Hons), University of Moratuwa
Senior Software Engineer / Lecturer

SINCE LAST
WEEK

Types were the foundation. Prove it.

- `go test` on `FullName` is green — including trim cases. If not, fix that first.
- Cold-call: zero value of `string`? Why is `len("café")` 5?
- `gofmt` still required. Unformatted PRs are returned.
- Today is dense. We will do **two labs**: `control+slices`, then `maps`.

TODAY **If Day 1 of week 3 works, you can**

- Write `if`, `switch`, and every shape of `for` / `range`.
- Tell an array from a slice; use `make`, `append`, `len`, `cap`.
- Explain the **backing-array gotcha** after `append`.
- Read and write maps with the comma-ok check.
- Write a function that returns `(value, error)` as a pattern you copy.
- Ship `Contains`, `Unique`, and a frequency counter with `green go test`.

**SOFTWARE
ENGINEERING**

Control flow is how requirements become behaviour

A requirement says “reject empty email” or “filter by course”. That is an `if`, a loop, and a collection — not a framework.

- SMS later: list students, keep those whose course matches, count by course.
- Today’s katas become those utilities. Write tests on paper first.
- Still no HTTP. Still no database. Language that ships product.

IF Conditions are boolean — nothing else

```
score := 72
if score ≥ 50 {
    fmt.Println("Score is high")
} else if score < 50 {
    fmt.Println("Score is low")
} else {
    fmt.Println("Score is unknown")
}
```

- No parentheses around the condition. Braces are required — even for one line.
- There is no `if score`. An `int` is not a `bool`.

IF Short statement — declare, then decide

```
if n := len(name) {  
    fmt.Println("name is", n)  
} else {  
    fmt.Println("name is not visible")  
}  
// n is not visible
```

The short statement runs first. The name lives only inside the if/else block.

You will see this endlessly with `if err := ...; err != nil` from week 5 on. Learn the shape now.

SWITCH

Many equal cases, one place

```
day := "Sat"
switch day {
case "Sat", "Sun":
    fmt.Println("weekend")
case "Mon", "Tue", "Wed":
    fmt.Println("weekday")
default:
    fmt.Println("unknown")
}
```

- No automatic fallthrough. Each case ends. Use `fallthrough` only if you mean it (rare).
- Cases can list several values with commas.

SWITCH

Tagless switch — a clean if/else if chain

```
score := 72
switch {
case score ≥ 90:
    fmt.Println("A")
case score ≥ 80:
    fmt.Println("B")
case score ≥ 70:
    fmt.Println("C")
default:
    fmt.Println("D")
}
```

- No expression after switch. Cases are boolean conditions.
- Prefer this over a tall else if tower when each branch is a clear rule.

FOR Go has one loop keyword

C-style

```
for i := 0; i < 5; i++ {  
    fmt.Println(i)  
}
```

While-style

```
n := 3  
for n > 0 {  
    fmt.Println(n)  
    n--  
}
```

Forever

```
for {  
    break // leave  
}
```

No while / do

Those words do not exist. for covers every shape.

RANGE

```
courses := []string{"SE", "CS", "EE", "ME", "IE", "CE", "AE", "DE", "FE", "GE", "HE", "IE", "JE", "KE", "LE", "ME", "NE", "OE", "PE", "QE", "RE", "SE", "TE", "UE", "VE", "WE", "XE", "YE", "ZE"}

for i, c := range courses {
    fmt.Println(i, c)
}

for _, c := range courses {
    fmt.Println(c) // i
}

for i := range courses {
    fmt.Println(i) // i
}
```

On a string, range yields runes (you saw this in week 2).

BREAK /
CONTINUE

Leave early, or skip this turn

```
for i := 0; i < 10; i++ {  
    if i%2 == 0 {  
        continue // skip  
    }  
    if i > 7 {  
        break // leave  
    }  
    fmt.Println(i)  
}
```

continue → next iteration. break → exit the innermost loop.

Labels exist for nested loops. You do not need them today.

LIVE DEMO CONTROL FLOW

Type this with me

```
package main
import "fmt"
func main() {
    grade := "B"
    switch grade {
    case "A", "B":
        fmt.Println("good")
    default:
        fmt.Println("work")
    }
    for i, r := range "Go" {
        fmt.Printf("%d %c\n", i, r)
    }
}
```

ARRAYS
VS
SLICES

Fixed size vs the list you will actually use

- An **array** has a fixed length in its type: `[3]int`.
- A **slice** is a view: pointer + length + capacity. Type is `[]int`.
- Almost all Go code uses slices. Arrays show up in hashes and rare APIs.

```
var seats [3]int
courses := []string{"SE
more := make([]string,
```

LEN /
CAP /
MAKE

How big is it, and how big can it grow without reallocating?

```
s := make([]int, 2, 5)  
fmt.Println(len(s), cap(s))
```

```
s = append(s, 9)  
fmt.Println(len(s), cap(s))
```

- len — how many elements you can index.
- cap — size of the backing array from the start of this slice.
- make([]T, length, capacity) — prefer when you know you will append a lot.

APPEND

Always take the result

```
names := []string{"Ada"}  
names = append(names, "Grace")  
names = append(names, "Edsger", "  
  
// wrong: append(names, "x") - y
```

- append may reuse the backing array or allocate a bigger one.
- Either way, the returned slice header is the one you must keep.
- Appending to a nil slice is fine. Zero value of []string is nil, and append works.

THE
GOTCHA

Two slices can share a backing array

```
a := []int{10, 20, 30, 40}
b := a[:2]           // len 2
b = append(b, 99)    // may grow
fmt.Println(a)       // oft
```

If cap(b) still has room, append writes into the shared array.

Fix when you need independence: c := append([]int(nil), a...) or copy into a new slice.

This is the week-3 idea they must take home — same weight as zero values in week 2.

LIVE
DEMO

SLICES

Watch the shared array

```
package main
import "fmt"
func main() {
    a := []int{10, 20, 30, 40}
    b := a[:2]
    fmt.Println("before", a, b, "cap(b)=", cap(b))
    b = append(b, 99)
    fmt.Println("after ", a, b)
}
```

MAPS

Lookup by key — the dictionary type

```
ages := map[string]int{
    "Ada": 36,
    "Grace": 85,
}
ages["Ken"] = 7
fmt.Println(ages["Ken"])
delete(ages, "Ken")
```

Type is `map[Key]Value`. Keys must be comparable (string, int, ... — not slices).

Zero value is `nil`. Reading a nil map returns the zero value. Writing panics — make first, or use a literal.

COMMA-
OK

Missing key is not an error — check yourself

```
v, ok := ages["Ada"]  
if !ok {  
    fmt.Println("not found")  
} else {  
    fmt.Println(v)  
}  
fmt.Println(ages["Missing"])
```

- If you only write `v := ages["x"]`, you cannot tell “missing” from “present with zero value”.
- Comma-ok is the idiom. Memorise it. You will use it in SMS and in every Go job.

RANGE
OVER MAP

Order is not promised

```
for name, age := range m {  
    fmt.Println(name, age)  
}  
for name := range m {  
    fmt.Println(name)  
}
```

Iteration order is randomised. Never write a test that requires a fixed map print order.

Need sorted keys? Collect keys into a slice, `sort.Strings`, then look up.

LIVE
DEMO

MAPS

Frequency counter — lab 2 preview

```
package main
import "fmt"
func main() {
    words := []string{"SE", "CS", "SE", "SE"}
    count := make(map[string]int)
    for _, w := range words {
        count[w]++
    }
    fmt.Println(count)
    n, ok := count["SE"]
    fmt.Println(n, ok)
}
```

Enough to survive week 5 — not the full theory

```
func Contains(list []string) bool {
    for _, v := range list {
        if v == want {
            return true
        }
    }
    return false
}
```

```
func FullName(first, last string) string {
    if first == "" {
        return "", fmt.Errorf("first name is required")
    }
    return first + " " + last
}
```

Name, parameters, results.
Multiple returns are normal.

return x, err is a
pattern you copy. Deep
error theory is week 9.

Variadic and closures — know they exist

```
func Sum(nums ...int)
    total := 0
    for _, n := range nums
        total += n
    }
    return total
}

fmt.Println(Sum(1, 2, 3, 4, 5))

double := func(n int) int {
    return n * 2
}

fmt.Println(double(21))
```

... means “any number of args”.
Inside, it is a slice.

A function value can be assigned to a name. Closures close over outer variables — light only.

**SOFTWARE
ENGINEERING**

Write the test table before the code

For Contains, fill this on paper first. That is design, not bureaucracy.

```
Input list  
["SE", "CS"]  
["SE", "CS"]  
[]  
["SE", "SE"]
```

- Then implement. Then go test. Red → green is still the ritual.
- Same habit later for every SMS handler edge case.

LAB 1 •
CONTROL +
SLICES

Contains and Unique

- Package slicekit with green tests.
- `Contains(list []string, want string) bool`
- `Unique(list []string) []string` — first occurrence wins, order preserved.
- Fill the paper test table first. Then code.

```
mkdir -p ~/epic-go/week03/slicekit  
cd ~/epic-go/week03/slicekit  
go mod init epiclearn/slicekit  
# slicekit.go + slicekit_test.go  
go test
```

LAB 2 ·
MAPS

Frequency counter with tests

- `Freq(words []string) map[string]int`
- Empty input → empty map (not nil panic).
- Assert counts with comma-ok in the test — do not compare map print order.
- Both labs must be green before you leave.

**LAB
GATE**

Done when all of this is true

- Worksheet: control-flow + slice gotcha + comma-ok cells filled
- Paper test table for Contains completed before coding
- `go test` green for Contains, Unique, and Freq
- You can explain why append on a subslice changed the original
- Commit and push. Show me the GitHub URL

**NEXT
SESSION**

Week 4 — modules, packages, SMS kickoff

- `go mod init` with meaning. `internal/` as a boundary.
- Project layout matching this repo's backend.
- First SMS milestone: empty module + folders + README.
- Bring green week-3 tests. Product work starts.

Questions